



**INTERNACIONALNI
UNIVERZITET U SARAJEVU**

Rentals and Sales Price Prediction Report

Introduction To Machine Learning

EE418

International University of Sarajevo

December 29, 2025

Students:

- Emre Arapcic-Uevak
- Mustafa Sinanovic

Professor:

Dr. Kanita Hadziabdic

Contents

1	Introduction to the Problem	4
2	Materials and Methods	4
2.1	Data Acquisition	4
2.2	Data Overview	5
2.3	Data pre-processing	6
2.3.1	Data Cleaning	6
2.3.2	Missing Values	16
2.3.3	Deeper look at data cleaning	19
2.4	Feature Engineering	22
2.4.1	Feature Selection	22
2.4.2	Feature Creation	23
2.4.3	Feature Scaling	25
2.5	Methods	25
2.5.1	Model Selection	25
2.5.2	Parameter Optimization	25
3	Results	27
3.1	Best models	27
3.1.1	Linear Regression	27
3.1.2	Random Forest	27
3.1.3	K-Nearest Neighbors	27
3.1.4	Support Vector Regression	28
3.1.5	Gradient Boosting	28
3.2	Evaluation of best models	28
3.2.1	Linear Regression	31
3.2.2	Support Vector Machine	32
3.2.3	Gradient Boosting	33
3.2.4	Random Forest	34
3.2.5	K-Nearest Neighbors	35
3.3	Model Selection	36
4	Conclusion	36
5	Distribution of work	37
6	Resources	37

List of Figures

1	Distribution of conditions with respect to ad_type	6
2	Distribution of municipalities with respect to ad_type	8
3	Distribution of property types with respect to ad_type	9
4	Distribution of heating types with respect to ad_type	10
5	Distribution of heating types with respect to ad_type	11
6	Distribution of levels with respect to ad_type	12

7	Distribution of price_numerics with respect to ad_type	13
8	Distribution of m^2 's with respect to ad_type	14
9	Distribution of rooms with respect to ad_type	15
10	Bar plot representation of missing features	16
11	Matrix plot representation of missing features	17
12	Heat map of missing feature	17
13	Combination of missing feature counts	18
14	Comparison of all and corrected apartments	19
15	Removed Apartments	20
16	Mislabeled apartment	20
17	Value Counts per distance	21
18	Value Counts per distance	21
19	Corrected Apartments	22
20	Municipality of Novi Grad Sarajevo	23
21	All bus stops in Canton Sarajevo	24
22	Predicted versus true apartment sales prices using Linear Regression	31
23	Predicted versus true apartment rental prices using Linear Regression	31
24	Predicted versus true apartment sales prices using Support Vector Machine	32
25	Predicted versus true apartment rental prices using Support Vector Machine	32
26	Predicted versus true apartment sales prices using Gradient Boosting	33
27	Predicted versus true apartment rental prices using Gradient Boosting	33
28	Predicted versus true apartment sales prices using Random Forest	34
29	Predicted versus true apartment rental prices using Random Forest	34
30	Predicted versus true apartment sales prices using K-Nearest Neighbors	35
31	Predicted versus true apartment rental prices using K-Nearest Neighbors	35
32	QR Code to our repository	37

List of Tables

1	Overview of features and their possible values	5
2	Value counts of condition by ad_type	6
3	Value counts of municipality by ad_type	8
4	Value counts of property_type by ad_type	9
5	Value counts of heating by ad_type	10
6	Value counts of equipment by ad_type	11
7	Level distribution	12
8	Price distribution	13
9	m^2 distribution	14
10	Rooms distribution	15
11	Best Linear Regression hyperparameters for apartment sale price prediction	27
12	Best Linear Regression hyperparameters for apartment rental price prediction	27
13	Best Random Forest hyperparameters for apartment sale price prediction	27
14	Best Random Forest hyperparameters for apartment rental price prediction	27
15	Best K-Nearest Neighbors hyperparameters for apartment sale price prediction	27
16	Best K-Nearest Neighbors hyperparameters for apartment rental price prediction	27
17	Best Support Vector Regression hyperparameters for apartment sale price prediction	28
18	Best Support Vector Regression hyperparameters for apartment rental price prediction	28

19	Best Gradient Boosting hyperparameters for apartment sale price prediction	28
20	Best Gradient Boosting hyperparameters for apartment rental price prediction	28
21	Cross-validation and test R^2 score for sales models	29
22	Cross-validation and test Negative Mean Absolute Error score for sales models	29
23	Cross-validation and test Negative Root Mean Square Error score for sales models	29
24	Cross-validation and test R^2 score for rents models	30
25	Cross-validation and test Negative Mean Absolute Error score for rents models	30
26	Cross-validation and test Negative Root Mean Square Error score for rents models	30
27	Work Distribution Table	37

1 Introduction to the Problem

A major problem a lot of people face is that they do not know how to price their apartments or how to check for the "*fairness*" (If the given price matches real expectation) of a given price.

This problem also extends to renting as sometimes people are not sure what they should or could put as their monthly rent price nor do tenants know if the given rentals are worth the given renting price. Of course this is taking into consideration that the tenant has other options.

Our goal for this project was to test out a suite of *hypothesis spaces* (machine learning model classes) to see if we could get out a *hypothesis* (model) that could accurately predict the prices of apartments that are either being sold or rented out.

2 Materials and Methods

2.1 Data Acquisition

We decided to first focus on the canton of Sarajevo as it was the best option for us to create a proof of concept for Bosnia and Herzegovina.

Since there is a lack of information regarding apartments that were sold and their features in Bosnia and herzegovina we decided to webscrape the information from popular website like:

- OLX.ba
- nekretnine.ba

however since the data presented on these two was different we found it easiest to work with apartments listed on the OLX.ba.

Once we finished scraping the data, we uploaded it to kagglehub as it would be much easier to just update the dataset on kagglehub and download it from kagglehub whenever we wish to retrain the model rather than just saving it locally.

The dataset is available here.

2.2 Data Overview

The data we were able to get from every entry from OLX.ba is as follows:

title The title of the post

url The url to the post

price_numeric The numeric price of the apartment (either the renting or selling price)

municipality The municipality the apartment is located in

rooms The number of rooms the apartment has

square_m2 The m^2 of an apartment

condition The condition of the apartment

ad_type If the apartment is listed for sale or for renting

property_type Apartments labeled as either "Stan" or "Apartman" (More about this later)

equipment Feature that tells us how furnished the apartment is

level The floor the apartment is located on

heating The type of heating the apartment is using

latitude The latitude coordinate of the apartment

longitude The longitude coordinate of the apartment

For more detail on the data's domains please refer to the Table 1

Feature	Possible values
title	Title of the post
url	url of the scraped post
price_numeric	Continuous (BAM)
municipality	One of Sarajevo municipalities
rooms	Integer ≥ 1.0
square_m2	Continuous (m^2)
condition	New Build, Good Condition, Renovated, Partially Renovated, Under Construction, Needs Renovation
ad_type	Sale, Rent
property_type	Stan, Apartman
equipment	Furnished, Semi-furnished, Unfurnished
level	Integer
heating	District Heating, Gas Heating, Electric Heating, Central Gas Heating, Central (Boiler Room), Wood Heating, Other
latitude and lonitude	The coordinates of the apartment

Table 1: Overview of features and their possible values

2.3 Data pre-processing

In this subsection we will cover some of the pre-processing steps that we have taken, such as:

- Data Cleaning
- Handling of missing values

2.3.1 Data Cleaning

Lets first take a look at our categorical data.

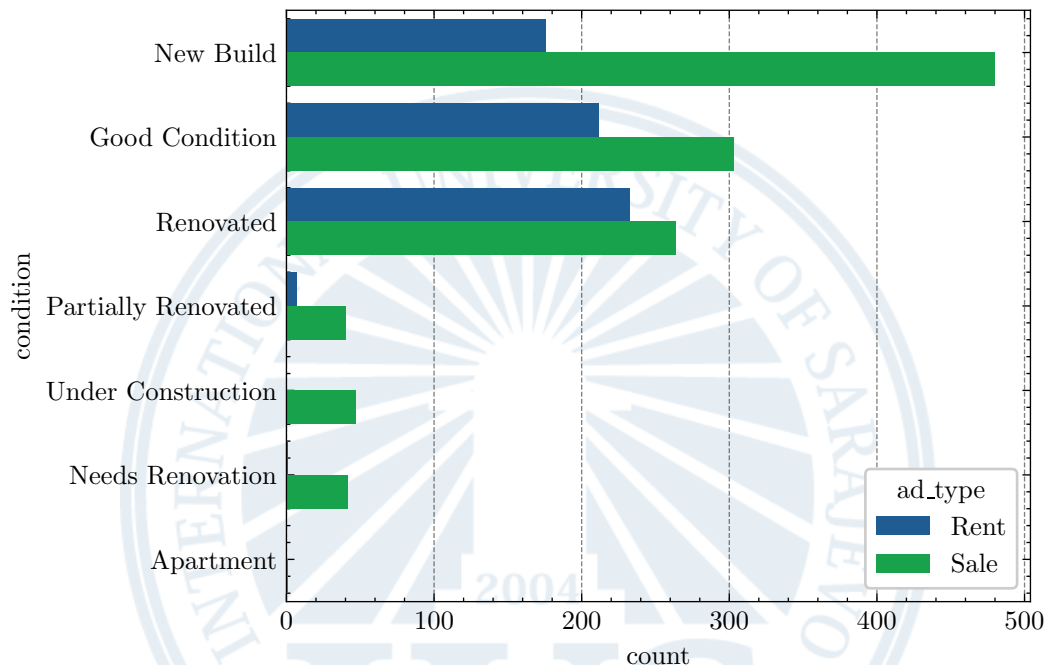


Figure 1: Distribution of conditions with respect to ad_type

Table 2: Value counts of condition by ad_type

condition	Count	condition	Count
Renovated	233	New Build	480
Good Condition	212	Good Condition	303
New Build	176	Renovated	264
Partially Renovated	7	Under Construction	47
Needs Renovation	1	Needs Renovation	42
Apartment	0	Partially Renovated	40
Under Construction	0	Apartment	1

(a) Value Counts for Rents

(b) Value Counts for Sales

As can be seen from the count plot, Fig 1, and the value counts tables, Table 2, there seems to be some mistakes in the data that need cleaning up.

The first thing that can be seen is the strange 'Apartment' value for the conditions which can only be found once for Sales, as can be seen in Table 2b. This is clearly a mistake made by the poster of the apartment and can be removed.

There is another interesting thing that can be seen in Figure 1, it is that rentals can never have the conditions:

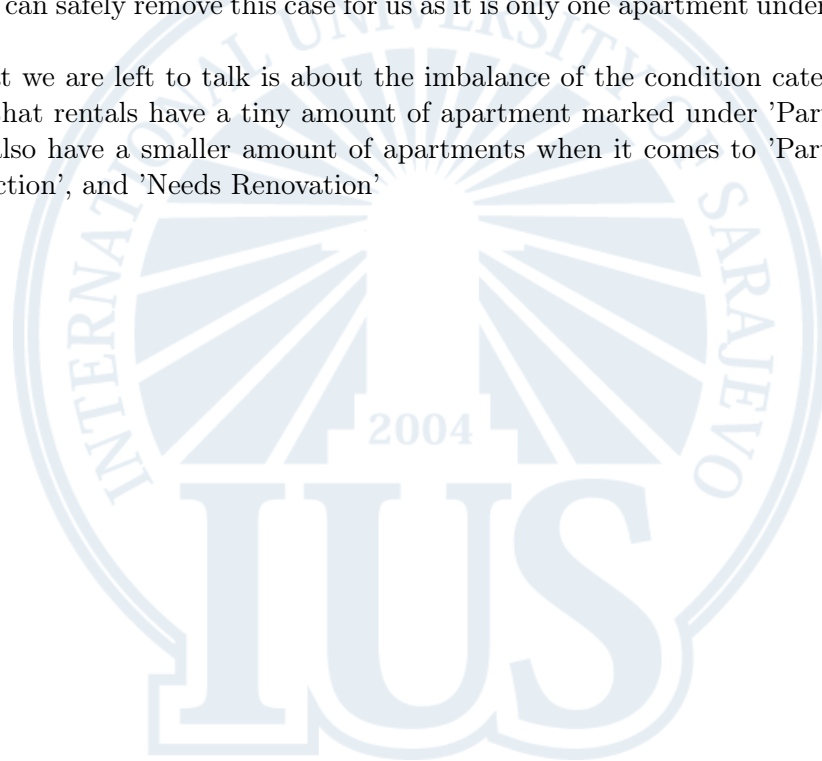
- Under Construction
- Needs Renovation

Which makes sense because you would not rent out an apartment that needs to be renovated and you cannot rent out an apartment that is under construction!

There is a caveat to this however, even though hard to see from the graph the table 2a does show one apartment marked as needing renovation but being listed up for renting.

However, we can safely remove this case for us as it is only one apartment under these conditions.

And finally what we are left to talk is about the imbalance of the condition category, mainly that as can be seen that rentals have a tiny amount of apartment marked under 'Partially Renovated', and that sales also have a smaller amount of apartments when it comes to 'Partially Renovated', 'Under Construction', and 'Needs Renovation'



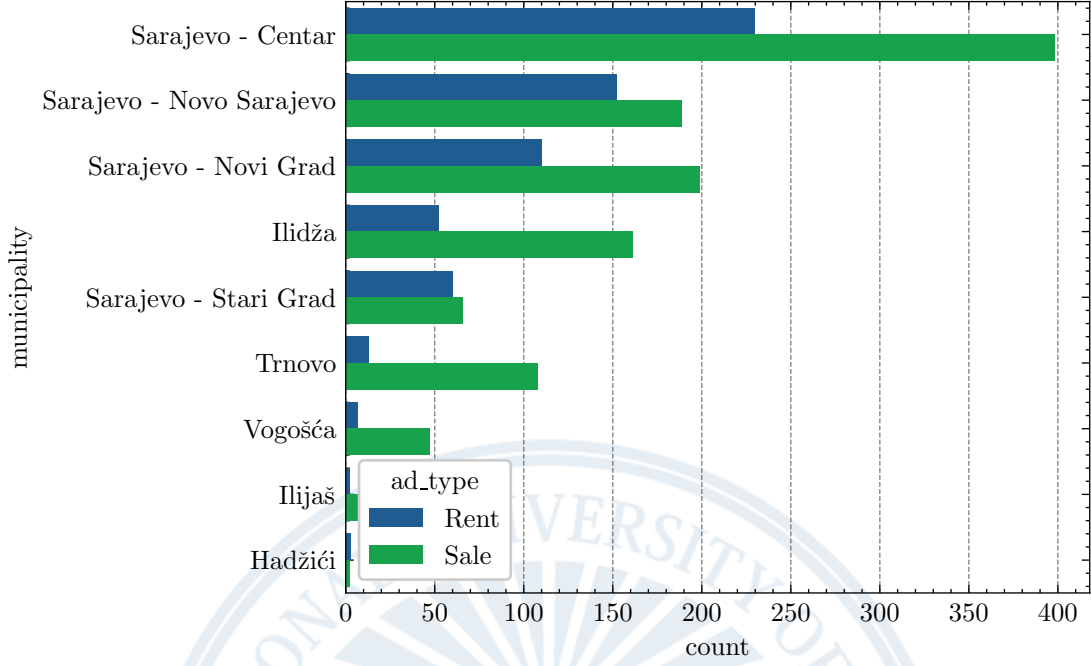


Figure 2: Distribution of municipalities with respect to ad_type

Table 3: Value counts of municipality by ad_type

municipality	Count	municipality	Count
Sarajevo - Centar	230	Sarajevo - Centar	398
Sarajevo - Novo Sarajevo	152	Sarajevo - Novi Grad	199
Sarajevo - Novi Grad	110	Sarajevo - Novo Sarajevo	189
Sarajevo - Stari Grad	60	Ilidža	161
Ilidža	52	Trnovo	108
Trnovo	13	Sarajevo - Stari Grad	66
Vogošća	7	Vogošća	47
Hadžići	3	Ilijaš	7
Ilijaš	2	Hadžići	2

(a) Value Counts for Rents
(b) Value Counts for Sales

As can be seen here, we do have a "shortage" of apartment from certain municipalities. However, this is not as problematic as it might seem, in the later sections where we talk about Feature Engineering and Feature Selection we will see that even a lot of the apartments that we have right now might not be usable, and this is all thanks to the fact that a lot of them do have coordinates or if they do they also have a chance to be either incorrectly placed or show locations of agencies.

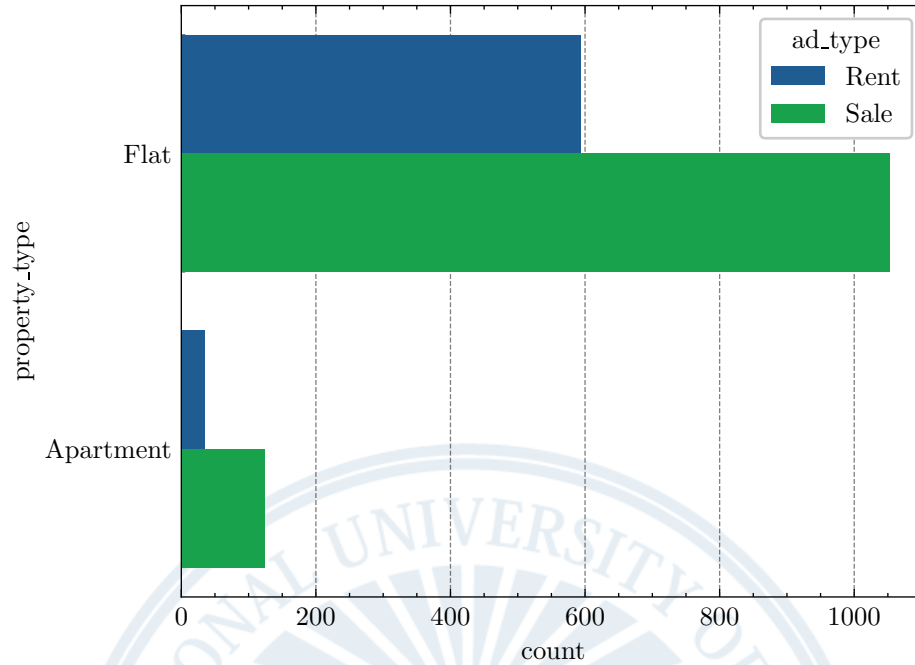


Figure 3: Distribution of property types with respect to ad_type

Table 4: Value counts of property_type by ad_type

property_type	Count
Flat	594
Apartment	35

(a) Value Counts for Rents

property_type	Count
Flat	1053
Apartment	124

(b) Value Counts for Sales

The graph and table shows us that there is generally a much larger amount of apartments that had their property type labeled as Flats rather than apartments but as we will talk about it during the Feature Selection section this feature seems redundant as it doesn't convey any additional information and the separation is not consistent.

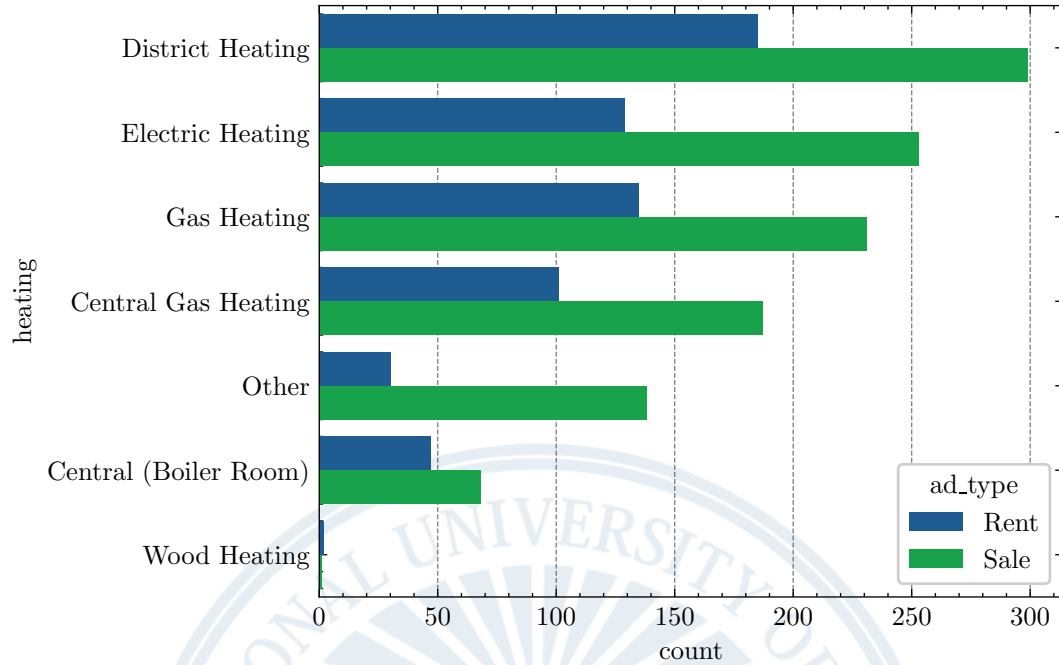


Figure 4: Distribution of heating types with respect to ad_type

Table 5: Value counts of heating by ad_type

heating	Count	heating	Count
District Heating	185	District Heating	299
Gas Heating	135	Electric Heating	253
Electric Heating	129	Gas Heating	231
Central Gas Heating	101	Central Gas Heating	187
Central (Boiler Room)	47	Other	138
Other	30	Central (Boiler Room)	68
Wood Heating	2	Wood Heating	1

(a) Value Counts for Rents

(b) Value Counts for Sales

Here we can also see that some categories of heating appear more often than others however the key insight here is the heating type "Wood Heating". It is problematically tiny.

However, there is a fix that we decided to go with; We took all the apartments with heating set to "Wood Heating" and re-categorized them as "Other"

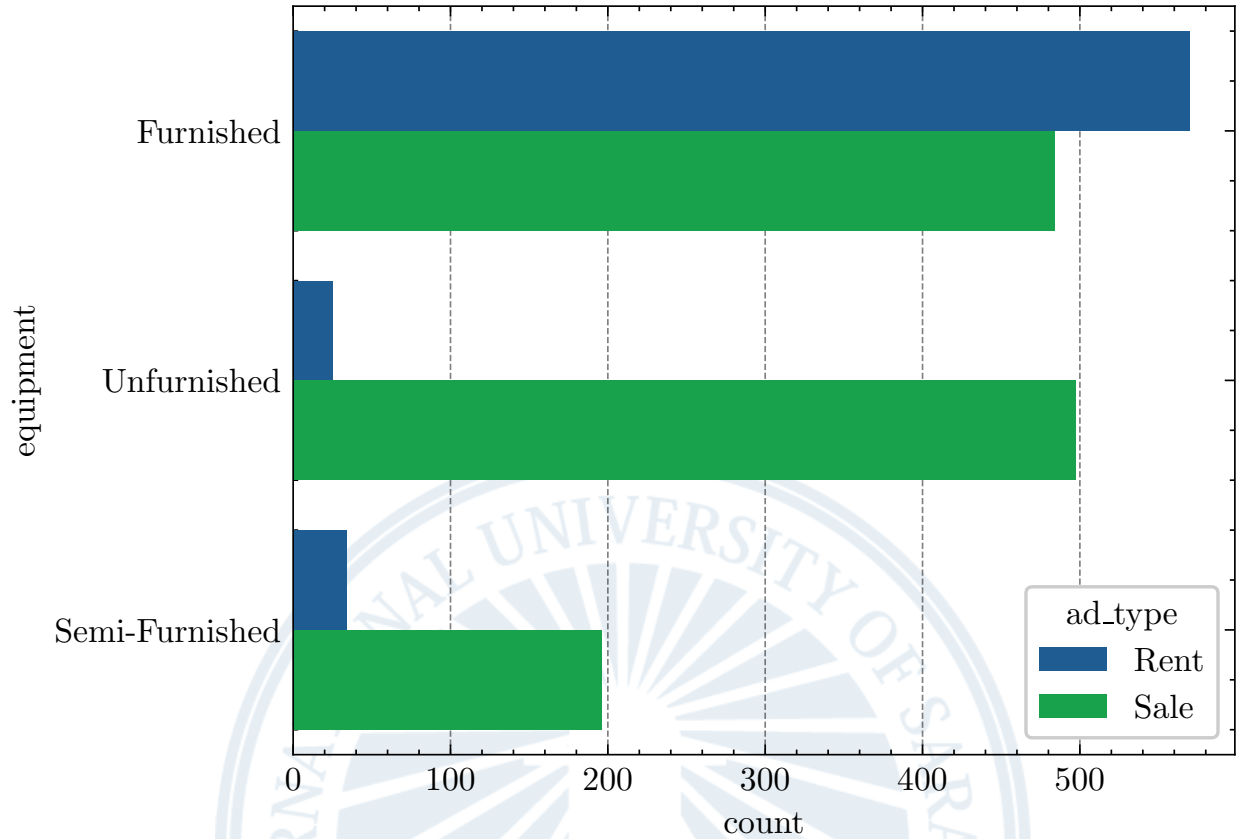


Figure 5: Distribution of heating types with respect to ad_type

Table 6: Value counts of equipment by ad_type

equipment	Count
Furnished	570
Semi-Furnished	34
Unfurnished	25

(a) Value Counts for Rents

equipment	Count
Unfurnished	497
Furnished	484
Semi-Furnished	196

(b) Value Counts for Sales

The most notable thing to see here is that Rentals have a tendency to be furnished more than anything. This makes sense as when are you to find an apartment listed up for rent that is not furnished.

This of course creates a big imbalance and will most likely make the 'equipment' feature less accurate for rental prices when it comes to "Unfurnished" and or "Semi-Furnished"

Now lets move to analysis of numeric features

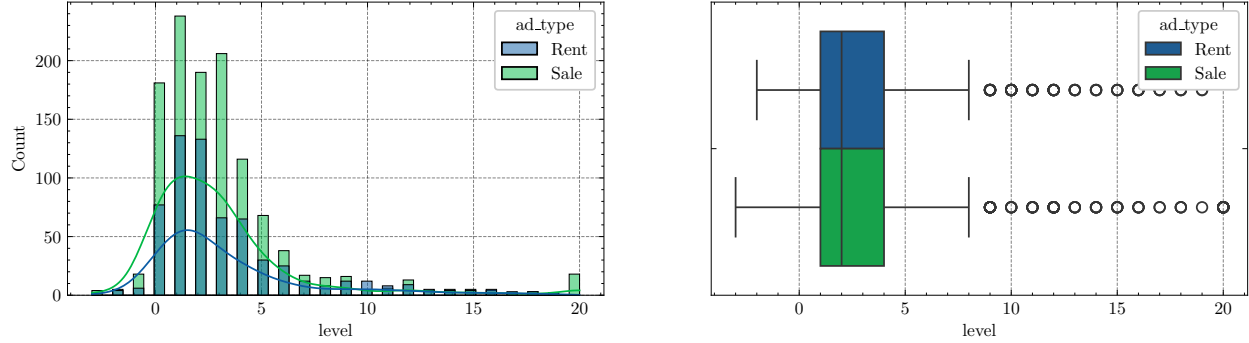


Figure 6: Distribution of levels with respect to ad_type

Table 7: Level distribution

Statistic	Value
count	629.00
mean	3.37
std	3.73
min	-2.00
25%	1.00
50%	2.00
75%	4.00
max	19.00

(a) Level Distribution of Rents

Statistic	Value
count	1177.00
mean	3.06
std	3.67
min	-3.00
25%	1.00
50%	2.00
75%	4.00
max	20.00

(b) Level Distributions for Sales

Apart from the distributions being equal for both rentals and sales there is one important information that cannot be gathered just from the plots themselves. Originally when we collected the data we have had an issue that levels were saving as *"Objects"* and not as Integers or Floats.

This happens because of the fact that on OLX.ba apartments can really only go up till floor 20, where anything above that is marked as *20+*. To spare the machine learning models from the so-called *curse of high dimensionality* we ended up converting all of the 20+ values into 20. The reason for us doing this was mainly because of our domain knowledge. We have had been looking around Sarajevo, as not only does it have the most of the listings in it but also has some of the highest buildings in it, we found out that most if not all of the bigger buildings hosting apartments went up to 21 and some up to 22 meaning that these apartments would be relatively close to one another; hence there is a spike that can be seen on the plot at Figure 6.

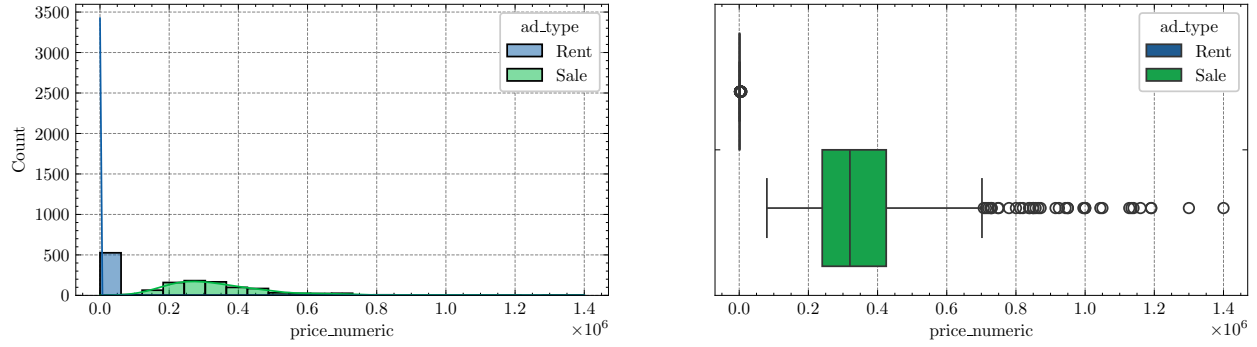


Figure 7: Distribution of price_numerics with respect to ad_type

Table 8: Price distribution

Statistic	Value
count	526.00
mean	1153.67
std	849.46
min	60.00
25%	650.00
50%	850.00
75%	1300.00
max	6000.00

(a) Level Distribution of Rents

Statistic	Value
count	902.00
mean	357497.83
std	177897.61
min	80000.00
25%	240000.00
50%	320000.00
75%	424975.00
max	1400000.00

(b) Level Distributions for Sales

Surprisingly the data here is clean and makes a lot of sense. The key idea here is not that there seems to be a lot of outliers as this is normal, as we will see when we take a look at the distribution of m^2 there are genuinely much larger apartments which will cost more, but the fact that the distribution differ so dramatically actually influenced us to make two separate machine learning models where one tries to predict the price for rentals and the other for sales.

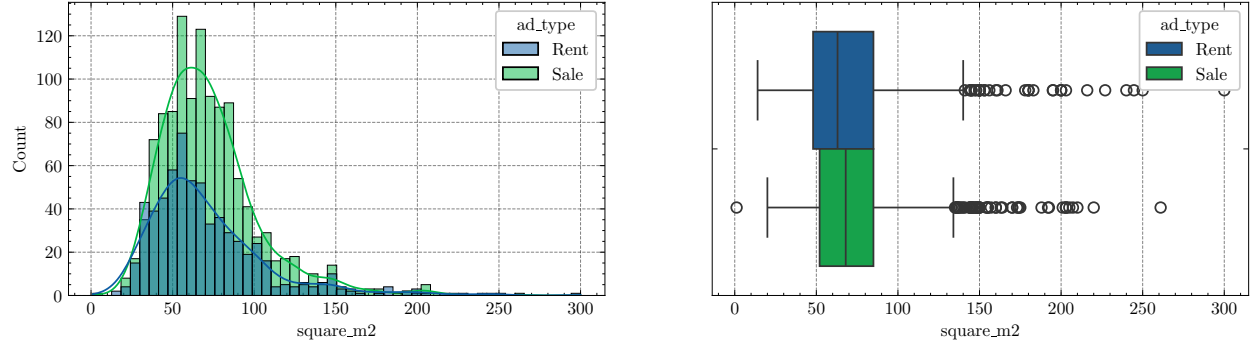


Figure 8: Distribution of m^2 's with respect to ad_type

Table 9: m^2 distribution

Statistic	Value
count	629.00
mean	71.18
std	36.88
min	14.00
25%	48.00
50%	63.00
75%	85.00
max	300.00

(a) m^2 Distribution of Rents

Statistic	Value
count	1177.00
mean	72.34
std	30.56
min	1.13
25%	52.00
50%	68.00
75%	85.00
max	261.00

(b) m^2 Distributions for Sales

Here as well the distribution seems okay, and as we can see there are some pretty giant apartments being both sold and rented which explains the previous price outliers.

However, one keynote here is the outliers for Sales; We find it very hard to believe that there is an apartment on sale that only has an area of $1.13m^2$, hence we ended up removing these outliers.

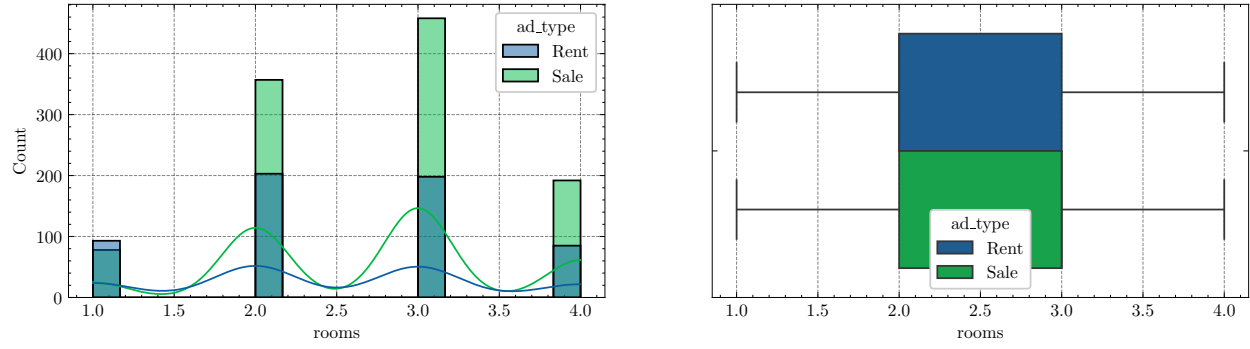


Figure 9: Distribution of rooms with respect to ad_type

Table 10: Rooms distribution

Statistic	Value
count	579.00
mean	2.47
std	0.93
min	1.00
25%	2.00
50%	2.00
75%	3.00
max	4.00

(a) Rooms Distribution of Rents

Statistic	Value
count	1085.00
mean	2.70
std	0.84
min	1.00
25%	2.00
50%	3.00
75%	3.00
max	4.00

(b) Rooms Distributions for Sales

And for our last analysis we have rooms. The only thing that we can see here is that so far we only have apartments with one to four rooms but no troubling data like some huge outliers nor values like half a room or no rooms at all.

2.3.2 Missing Values

Let's first plot out to see what amounts of missing features do we exactly have in our dataset that is currently missing, and then we will try to see if there are any correlations between features that are missing.

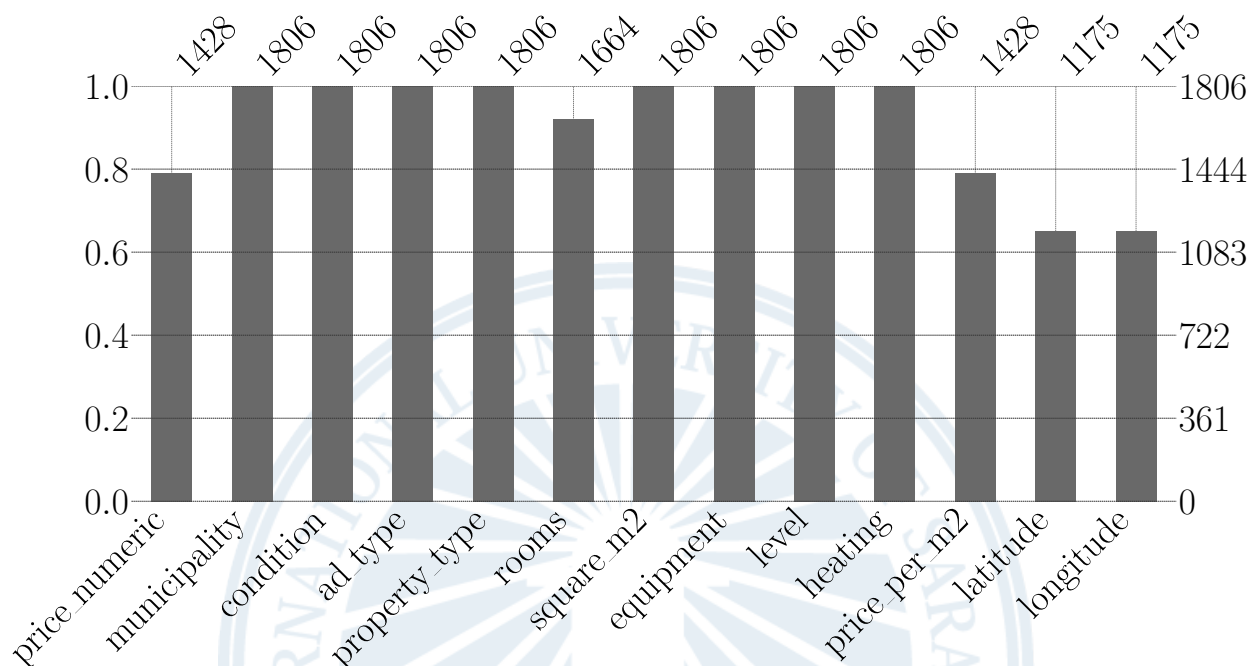


Figure 10: Bar plot representation of missing features

As we can see from Figure 10 we do have around 1806 entries, however there are certain things that we just cannot work with. First if we take a look at the price, which is our *target variable*, we will see that we only have 1428. This is to be expected as one of the issues we were trying to solve was the fact that some listings do not have a price. There is nothing that we can really do about the missing prices as imputing them will add unwanted bias.

Another issue with imputation of the prices is the fact that the distributions of features of apartments with and without the prices are not the same for every feature, meaning they don't follow the exact same trend. Using something like K-NN imputer could work but this would cause data leakage as we would have to impute looking at the entire dataset.

Another thing we have missing is the number of rooms. Luckily for us, we do not have a lot of apartments with missing rooms and unlike prices we can actually impute these with relatively low risk. However, there is one issue and it has to do with the next variable.

Another concern, apart from prices, is actually the coordinates. As can be seen we only have 1175 apartments that have coordinate, and we still don't know how many of the apartment with coordinate have prices. To be sure let's plot a matrix plot of the missing values and a heatmap to see if there could be any correlation between them.

Figure 12 tells us something we kind of expected to see. As soon as *latitude* is missing *longitude* is missing as well. This makes sense since the way coordinate were collected if there was no google map widget found neither latitude or longitude was collected.

The other non-surprising fact was that when *price* was absent *price_per_m2* was also absent which makes sense.

And Finally there seems to be a slight negative correlation between *rooms* and the coordinates, as if coordinates are missing there is a slightly higher chance that we will have the value for *rooms*.

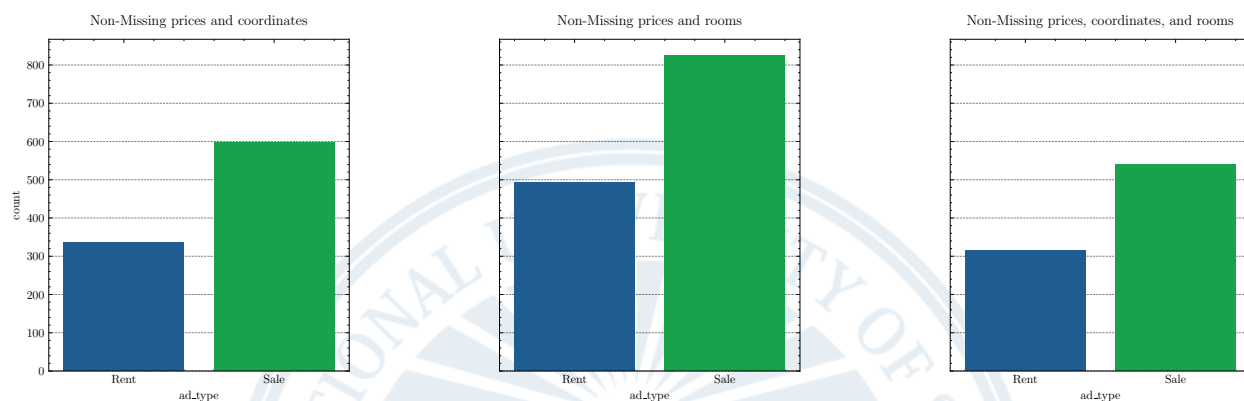


Figure 13: Combination of missing feature counts

As can be seen in Figure 13 We will lose a lot of apartments if we take out the apartments with no coordinates. This made us think of dropping the coordinates as we know that a lot of public models do not utilize coordinates. However, we will see in section 2.4.1 why we cannot drop this feature.

2.3.3 Deeper look at data cleaning

Before we can conclude this section there is another important thing that we need to talk about, and it concerns the coordinates. When we took a look at some of the apartments with coordinates we plotted them onto a map of the Sarajevo Canton, because we took the listings only from that region, and got the following image.

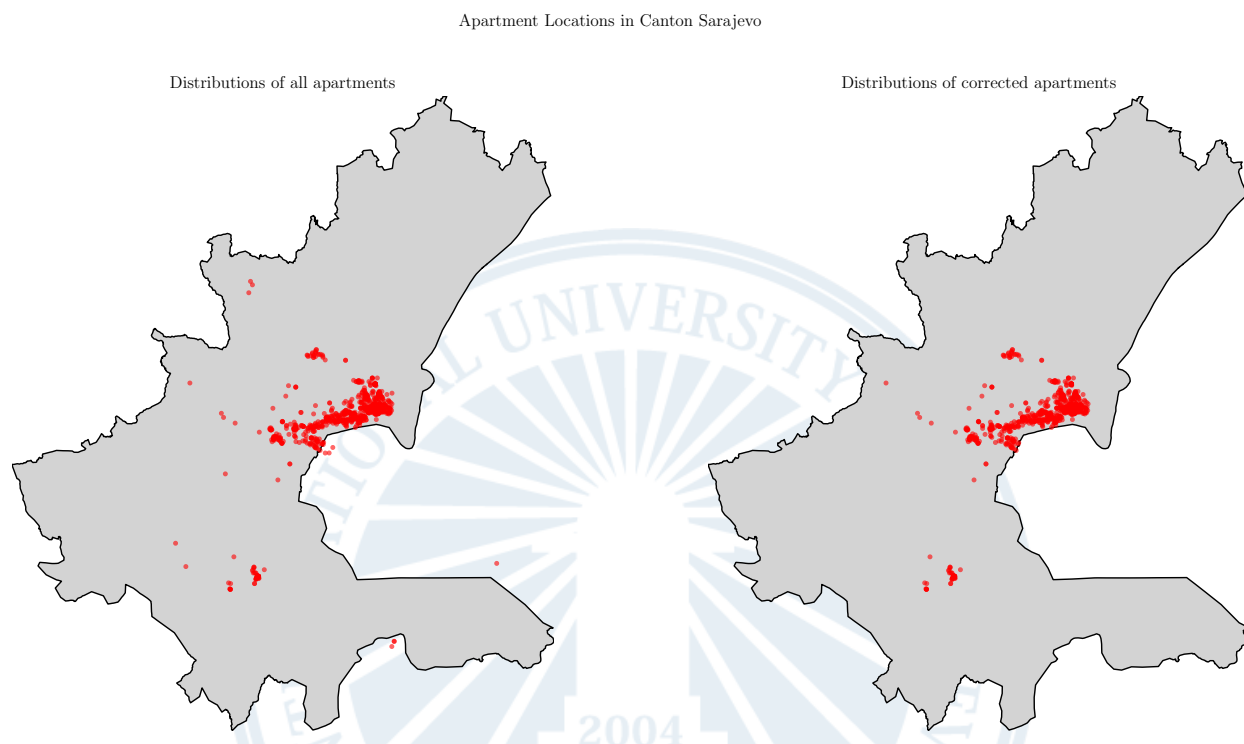


Figure 14: Comparison of all and corrected apartments

In Figure 14 we can see on the left a plot of all of the apartments zoomed in onto the Canton Sarajevo, and on the right we can see all the corrected apartments. But what does corrected apartments mean?

As can be seen from the Figure 14 there are some apartments that lie outside the Canton Sarajevo's boundary. In the given figure we can see that some apartments do fall outside of the cantons boundaries but there is something we cannot see.

Some of the apartments we have in our dataset are positioned in Saudi Arabia, Banja Luka, Vienna, ... etc. These are clearly either mistakes, or what is more likely is for them to be locations of agencies so we ended up removing these apartments.

However, this is not the only problem that there is. If we take a look at the apartments inside the canton there still can be miss-classified and or miss-labeled apartments. The way we checked for this this and fixed it was by looking in which municipality the apartment was located at, compared to what it was actually labeled as. This gave us outliers that were either human error or possible agencies selling apartments further from them.

Figure 15 shows all the apartments that were removed because they were not in municipalities that they were advertised to be. And, as can be seen, this seems to have worked. A lot of apartments that were removed were positioned kilometers from the closest boundary of their given municipality

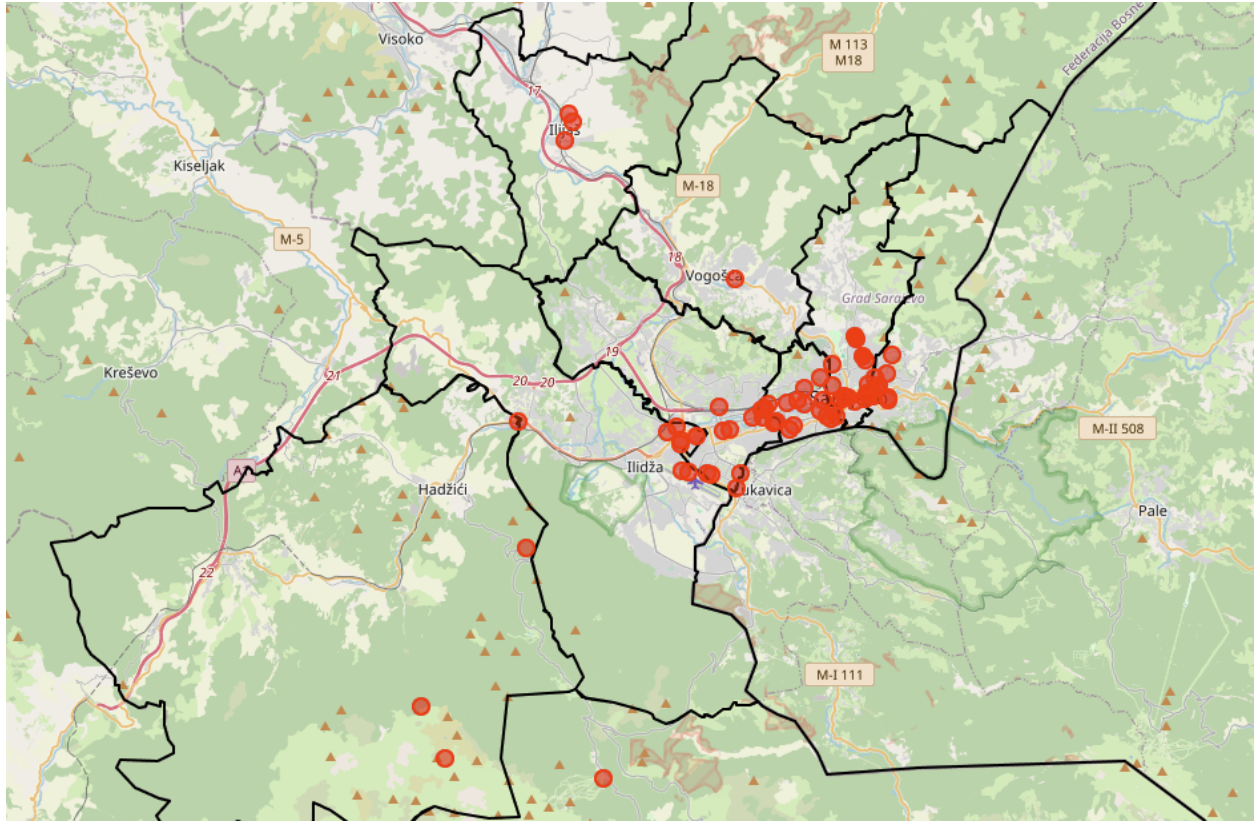


Figure 15: Removed Apartments

which highly indicated that these locations were actually locations of agencies.

However, this is not the only case. There are actually some apartment that lie really close to the border which are miss labeled, and this is most certainly due to human error. Figure ?? shows exactly this case, this apartment was labeled to be in the municipality of Ilidža but was actually in municipality of Hadžići. But the apartment lies 9 meters away from the boundary. This can be because of the error of coordinates not being precise enough, or it could be human error but whatever the case is these apartments should be added back in.

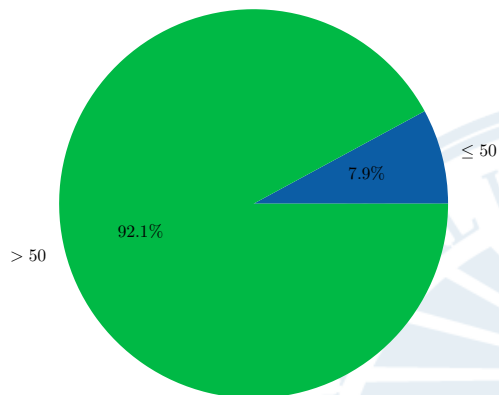


Figure 16: Mislabeled apartment

Okay now we know that we should add apartments that were mislabeled close to their border, but the question is how far can an apartment be to the border for us to label it as human or precision error? In other words how far a given apartment be from their claimed municipality until it gets removed?

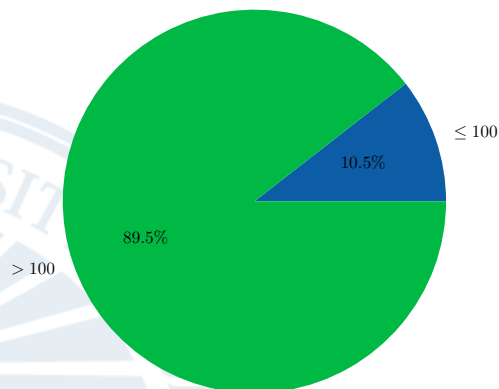
Well there is no hard given answer, we know for a fact that if it is really far off, we are talking kilometer and up, that it should get removed. To see how many apartments we captured with distances we decided to plot the result for distances: 50, 100, 200, and 300 meters.

Distributions of removed apartments close to border



(a) value counts of apartments 50m away

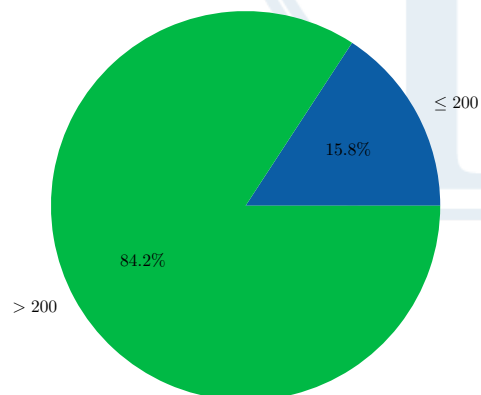
Distributions of removed apartments close to border



(b) value counts of apartments 100m away

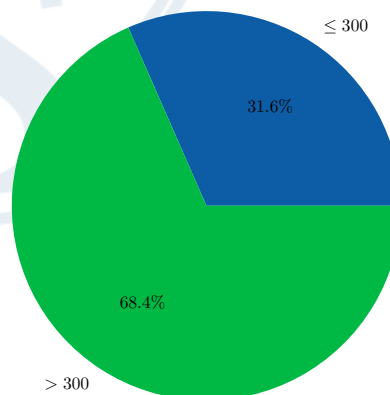
Figure 17: Value Counts per distance

Distributions of removed apartments close to border



(a) value counts of apartments 200m away

Distributions of removed apartments close to border



(b) value counts of apartments 300m away

Figure 18: Value Counts per distance

Since the jumps between 50, 100, and 200 are small, and the only major jump comes up when we check for apartments 300m away from the borders, we decided to go with those apartments, and they did provide the best results. The apartments that we decided to add back in can all be seen in Figure 19

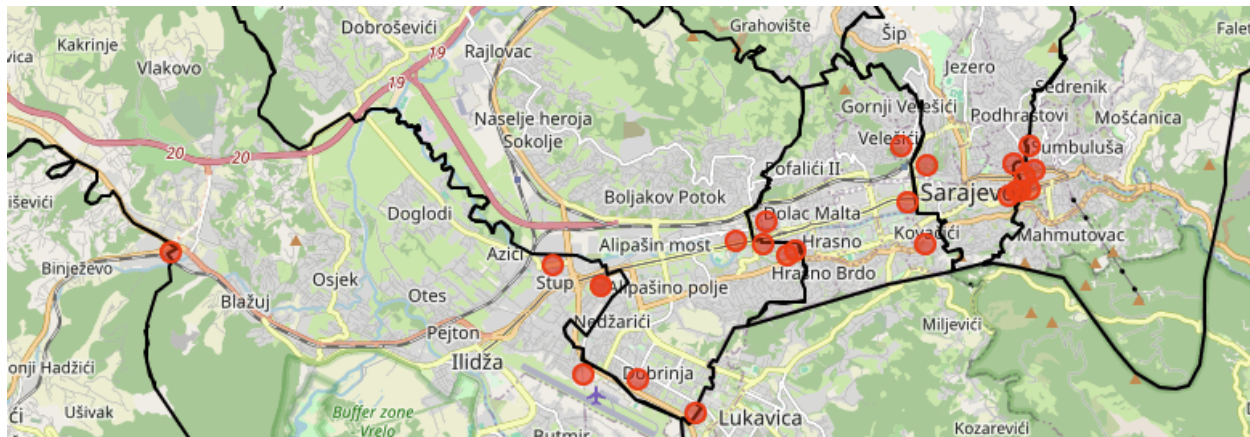


Figure 19: Corrected Apartments

We ended up just changing the municipalities of these apartments to their actual municipality that they were located in and added them back to the dataset.

2.4 Feature Engineering

2.4.1 Feature Selection

When it comes to feature selection we do not have a lot of issues. Looking back at Table 1 we can immediately see that we do not need things like 'title' and 'url'.

As we talked about in section 2.3.1 the feature 'property_type' is not that useful and keeping it will cause more trouble than

The harder question that we asked ourselves was if we wanted to keep the coordinates (longitude and latitude) or not, since a lot of apartments would be removed if we were to get rid of coordinates as there was no way to impute the coordinates. And to answer this question we looked back about what we said about the 'municipality' feature; when thinking of how someone would price their own home we thought, okay if the houses are approximately in the same location, look the same, and so on, that their prices would be close.

However, the main issue with this was the sheer fact that without coordinates we only had to rely on the municipality feature for the general location and other available features on hand. The problem? Municipality is not a good indicator of location-based price variation. Figure 20 shows a municipality that contains multiple apartments equal when it comes to features, but different when it comes to prices.

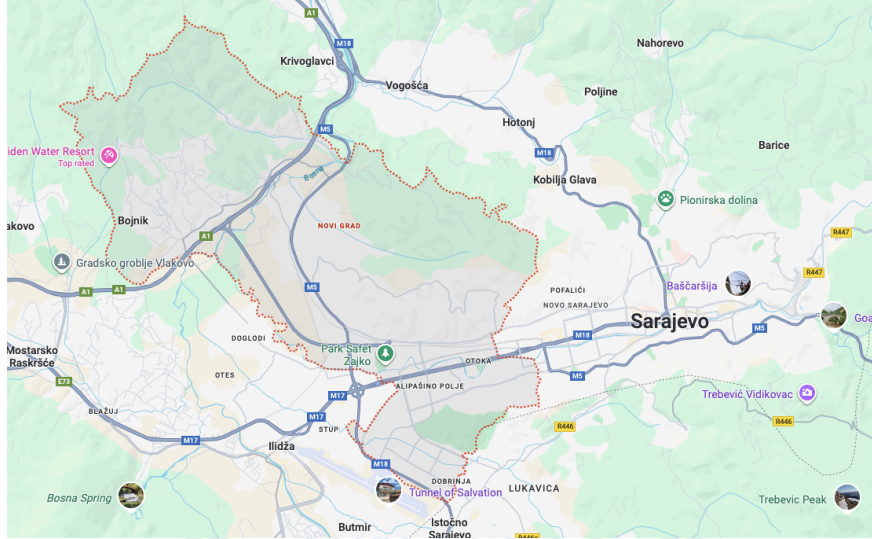


Figure 20: Municipality of Novi Grad Sarajevo

And in practice when we tried to train models on just these features themselves the models performed horribly. On data that models never saw before the best values we got were all around $R^2 = 0.12$! And this made us realize that coordinates had to stay.

Whether we should keep municipalities or not as features, we decided to stratify the train and test split using the municipality feature as this would make sure that we got a nice sample of apartments around the entire canton, however when it came to keeping it for training we did some testing and models performed way better without it then with it.

Another important thing to mention is that the models themselves also did feature selection as a part of hyperparameter tuning.

2.4.2 Feature Creation

When it comes to feature creation we had to find a way to create/extract some useful data from the coordinates of the apartments, such that our machine learning model could use said new features to find the best hypothesis.

*What we ended up doing was, utilizing the **OSMNX** (Open Street Map Network X) library, collect the following nodes:*

- *hospitals*
- *clinics*
- *pharmacies*
- *schools*
- *universities*
- *kindergartens*

- *bus stops*
- *tram stop*
- *train stations*
- *parking's*
- *parks*
- *playgrounds*
- *malls*
- *supermarkets*
- *marketplaces*
- *restaurants*
- *cafés*
- *places of worships*
- *police stations*
- *fire stations*
- *post offices*
- *sport centers*
- *fitness centers*
- *stadiums*

Then we found the closest one of each of these nodes to an apartment, and add a new feature "*closest_X_m*".

For example for bus stops, each apartment would get a new feature called the "*closest_bus_stop_m*" which was the distances of the closest bus stop from the apartment in meters.

Distribution of bus_stops

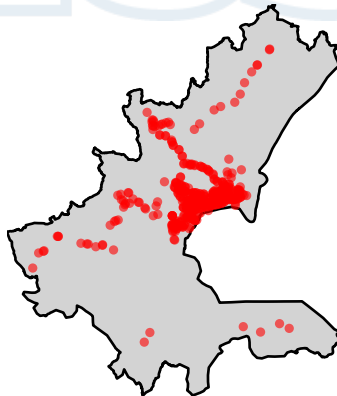


Figure 21: All bus stops in Canton Sarajevo

2.4.3 Feature Scaling

Since we ended up using models that were sensitive to the values of features, such as K -NN being sensitive to values with higher values than others because of it using the distance metric, we decided to scale the numeric feature using the Standard Scaler.

2.5 Methods

2.5.1 Model Selection

When it comes to model selection we used scikit's default parameter models to quickly get an overall image of how well these models would perform, of course the selection of these models was based on our knowledge of how they worked and the knowledge about the problem domain.

The models were quickly trained on the training data and were then compared to the "base" error, the error the model makes if it predicts that all the apartments have the same mean price, and after trying all of them out the best performing models were:

- Random Forest Regressor
- Linear Regression
- K Nearest Neighbors Regressor
- Support Vector Machines
- Gradient Boosting Regressor

2.5.2 Parameter Optimization

For each model class introduced in Section 2.5.1, an extensive hyperparameter search was conducted using a predefined parameter grid. The goal of this procedure was not only to improve predictive performance, but also to evaluate the sensitivity of each model to its most influential hyperparameters.

All hyperparameter combinations were evaluated using cross-validation on the training set. In addition to the standard model formulations, we also evaluated variants where the target variable was transformed using a logarithmic mapping via a **TransformedTargetRegressor**. This was motivated by the strong right-skewness typically observed in real estate price distributions (Figure 7).

Random Forest Regressor For the Random Forest model, we explored a wide range of structural and regularization-related hyperparameters. Specifically, the number of trees was varied to balance model stability and computational cost, while tree depth and minimum sample thresholds were adjusted to control overfitting. Both standard Random Forest models and logarithmically transformed target variants were evaluated.

- Number of trees: {50, 100, 200, 500}
- Maximum tree depth: {None, 2, 3, 5, 10, 20, 30}
- Minimum samples per leaf: {1, 5, 10, 20}
- Minimum samples for split: {2, 10, 20}
- Feature selection strategy: {None, sqrt, log2}

Linear Regression For Linear Regression, the focus was placed on feature selection rather than model regularization, as the underlying estimator is deterministic. A univariate statistical feature selection method based on the F-test was employed to study the impact of dimensionality reduction. Both raw and log-transformed target variants were considered.

- Feature selection method: `SelectKBest` with F-statistic
- Number of selected features: $\{2, 3, 5, 10, 20, 30, \text{all}\}$

K-Nearest Neighbors Regressor For the KNN model, hyperparameters governing neighborhood size, distance metrics, and weighting strategies were explored. Given the sensitivity of KNN to scale and noise, both standard and transformed-target variants were evaluated.

- Number of neighbors: $\{1, 3, 5, 10, 20, 30, 50, 100\}$
- Weighting scheme: $\{\text{uniform}, \text{distance}\}$
- Search algorithm: `ball_tree`, `kd_tree`, `brute`, `auto`
- Distance metric: $p \in \{1, 2\}$ (Manhattan and Euclidean)

Gradient Boosting Regressor Histogram-based Gradient Boosting was chosen due to its efficiency on tabular data. The hyperparameter grid focused on tree depth, learning rate, and the number of boosting iterations. As with previous models, both standard and log-transformed target variants were examined.

- Maximum tree depth: $\{3, 5, 7\}$
- Learning rate: $\{0.05, 0.1\}$
- Number of boosting iterations: $\{200, 400\}$

Support Vector Regression For Support Vector Regression, both linear and radial basis function (RBF) kernels were evaluated. The regularization strength, ϵ -insensitive loss width, and kernel parameters were systematically varied. Log-transformed target variants were included to mitigate sensitivity to extreme price values.

- Regularization parameter C : $\{0.1, 1, 10, 100\}$
- ϵ -insensitive loss: $\{0.01, 0.1, 0.2, 0.5\}$
- Kernel type: `linear`, `rbf`
- RBF kernel coefficient γ : $\{\text{scale}, 0.01, 0.1, 1\}$

After completing the hyperparameter search, the best-performing configuration from each model class was selected based on cross-validation performance and subsequently compared on a held-out test set. For every model mentioned in the section 2.5.1 we did hyperparameter (fine) tuning. After which we took all the best performing models from each model class and compared them to one another.

3 Results

3.1 Best models

In this section we will look at the parameters of the models that have had the best CV R^2 score.

3.1.1 Linear Regression

Table 11: Best Linear Regression hyperparameters for apartment sale price prediction

Best CV Score	feature_selector	k
0.674	SelectKBest (f-regression)	20.000

Table 12: Best Linear Regression hyperparameters for apartment rental price prediction

Best CV Score	feature_selector	k
0.421	SelectKBest (f-regression)	30.000

3.1.2 Random Forest

Table 13: Best Random Forest hyperparameters for apartment sale price prediction

Best CV Score	min_samples_leaf	min_samples_split	n_estimators
0.720	1.000	2.000	50.000

Table 14: Best Random Forest hyperparameters for apartment rental price prediction

Best CV Score	max_depth	min_samples_leaf	min_samples_split	n_estimators
0.527	10.000	1.000	2.000	100.000

3.1.3 K-Nearest Neighbors

Table 15: Best K-Nearest Neighbors hyperparameters for apartment sale price prediction

Best CV Score	algorithm	n_neighbors	p	weights
0.675	ball_tree	5.000	1.000	distance

Table 16: Best K-Nearest Neighbors hyperparameters for apartment rental price prediction

Best CV Score	algorithm	n_neighbors	p	weights
0.475	ball_tree	5.000	2.000	distance

3.1.4 Support Vector Regression

Table 17: Best Support Vector Regression hyperparameters for apartment sale price prediction

Best CV Score	C	epsilon	gamma	kernel
0.725	100.000	0.100	scale	rbf

Table 18: Best Support Vector Regression hyperparameters for apartment rental price prediction

Best CV Score	C	epsilon	gamma	kernel
0.544	100.000	0.100	0.100	rbf

3.1.5 Gradient Boosting

Table 19: Best Gradient Boosting hyperparameters for apartment sale price prediction

Best CV Score	learning_rate	max_depth	max_iter
0.727	0.100	3.000	400.000

Table 20: Best Gradient Boosting hyperparameters for apartment rental price prediction

Best CV Score	max_depth	learning_rate	max_iter
0.581	7.000	0.100	400.000

3.2 Evaluation of best models

Now that we have the best models of each model class we need a way to choose the best performing one of them all.

To do this we will check scores like:

- 1. Root-Mean-Square-Error (RMSE)*
- 2. Mean-Absolute-Error (MAE)*
- 3. R^2 score*

On both train, using cross validation, and test sets. Our goal is to see how well the models generalize and to get the average error the model makes as low as possible.

Model	CV_R2	Test_R2
SVR	0.725	0.810
GradientBoostingRegressor	0.727	0.806
RandomForest	0.682	0.783
LinearRegression	0.674	0.769
KNeighborsRegressor	0.675	0.762

Table 21: Cross-validation and test R^2 score for sales models

Model	CV_NMAE	Test_NMAE	Baseline_NMAE
SVR	-56652.934	-46456.333	-102582.692
GradientBoostingRegressor	-52631.780	-45336.833	-102582.692
RandomForest	-55214.976	-48075.361	-102582.692
LinearRegression	-61555.003	-55003.266	-102582.692
KNeighborsRegressor	-62187.245	-55960.469	-102582.692

Table 22: Cross-validation and test Negative Mean Absolute Error score for sales models

Model	CV_NRMSE	Test_NRMSE	Baseline_NRMSE
SVR	-79940.710	-63291.713	-145130.067
GradientBoostingRegressor	-79037.392	-63982.210	-145130.067
RandomForest	-87347.946	-67672.401	-145130.067
LinearRegression	-86950.603	-69762.958	-145130.067
KNeighborsRegressor	-86116.722	-70804.494	-145130.067

Table 23: Cross-validation and test Negative Root Mean Square Error score for sales models

Model	CV_R2	Test_R2
KNeighborsRegressor	0.475	0.696
RandomForest	0.509	0.675
GradientBoostingRegressor	0.581	0.606
LinearRegression	0.421	0.445
SVR	0.544	0.404

Table 24: Cross-validation and test R^2 score for rents models

Model	CV_NMAE	Test_NMAE	Baseline_NMAE
KNeighborsRegressor	-362.061	-289.253	-550.323
RandomForest	-328.944	-297.516	-550.323
GradientBoostingRegressor	-320.879	-294.845	-550.323
LinearRegression	-353.536	-331.456	-550.323
SVR	-335.007	-348.715	-550.323

Table 25: Cross-validation and test Negative Mean Absolute Error score for rents models

Model	CV_NRMSE	Test_NRMSE	Baseline_NRMSE
KNeighborsRegressor	-527.576	-422.298	-766.334
RandomForest	-510.557	-437.167	-766.334
GradientBoostingRegressor	-470.090	-481.294	-766.334
LinearRegression	-550.006	-570.910	-766.334
SVR	-498.346	-591.682	-766.334

Table 26: Cross-validation and test Negative Root Mean Square Error score for rents models

As we can see from table 21 to table 26. Scores on the model for sales scored significantly better than the model for rents. This is because after doing all of the data cleaning, as mentioned in the section 2.3.1, we are left with 498 apartments for sales, and only 287 apartments for rents! This is a really low number, especially when taking into account that we now have 30 features.

Before we select one of these models, let's first plot out the models predictions on the test data versus the ground truth values of the test data for every one of them.

3.2.1 Linear Regression

Predicted vs True Values for LinearRegression [Sales]

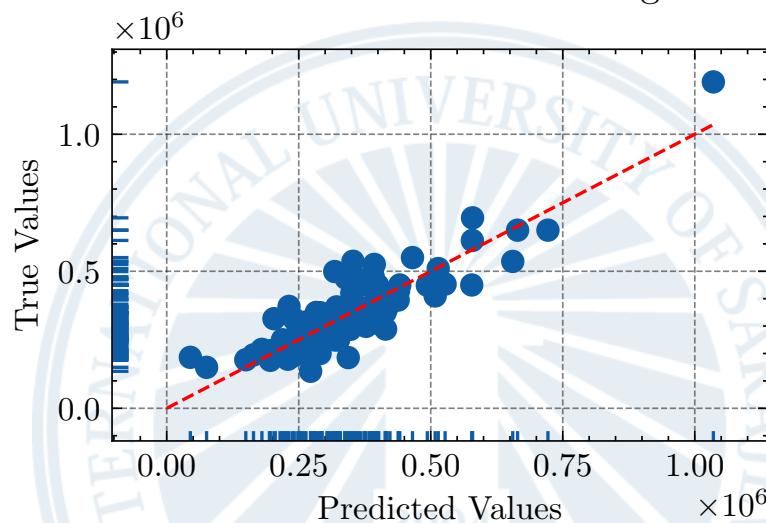


Figure 22: Predicted versus true apartment sales prices using Linear Regression

Predicted vs True Values for LinearRegression [Rents]

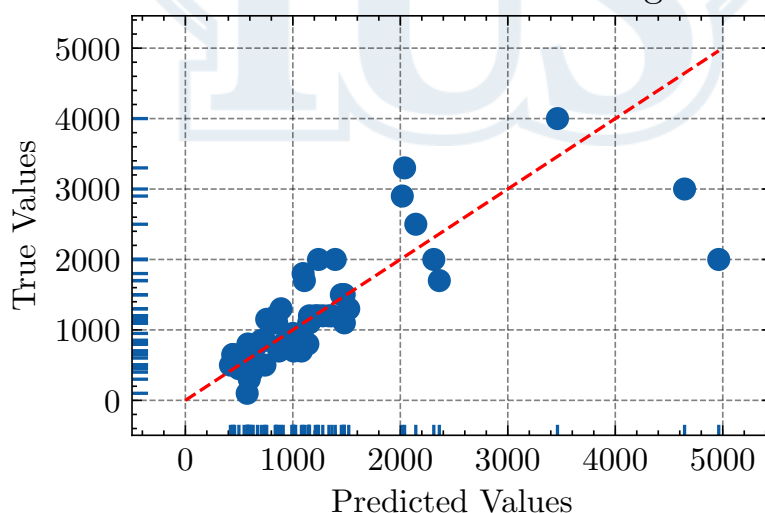


Figure 23: Predicted versus true apartment rental prices using Linear Regression

3.2.2 Support Vector Machine

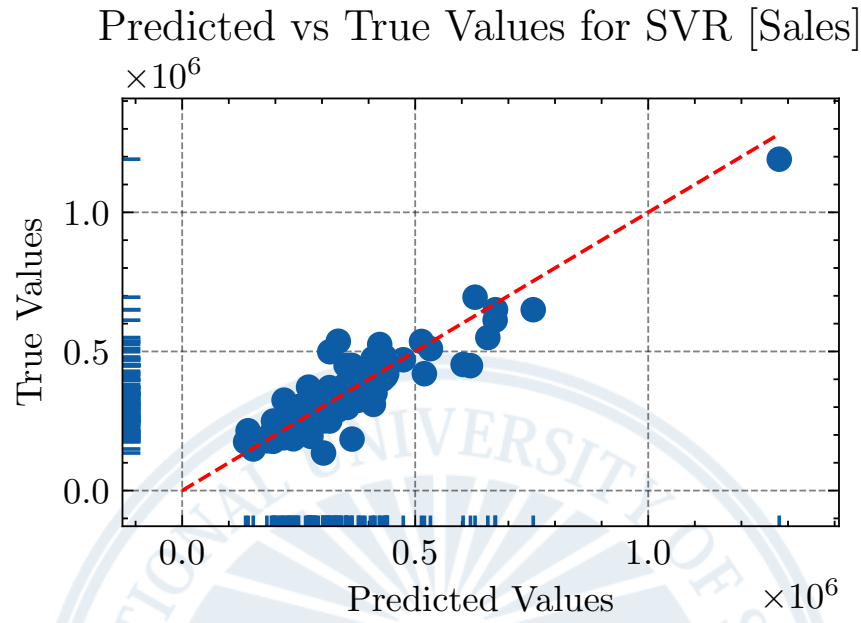


Figure 24: Predicted versus true apartment sales prices using Support Vector Machine

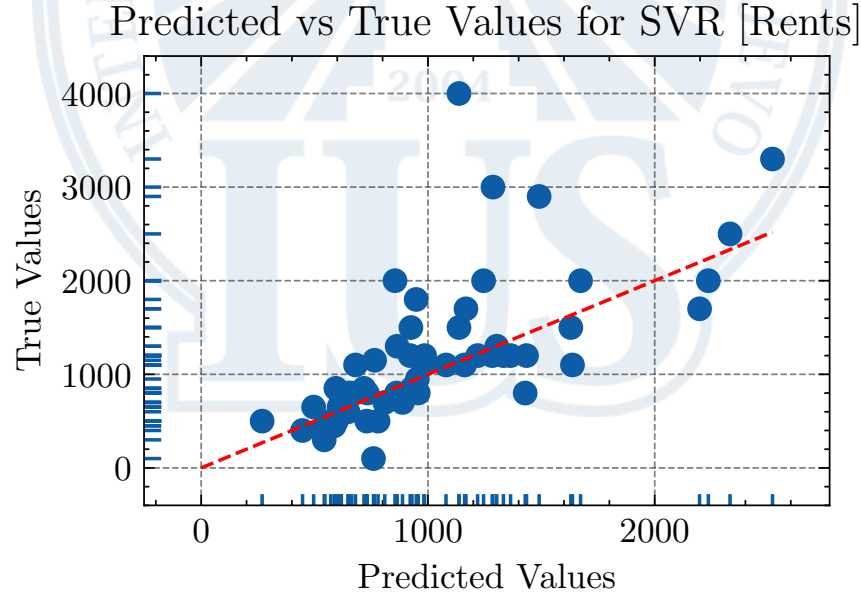


Figure 25: Predicted versus true apartment rental prices using Support Vector Machine

3.2.3 Gradient Boosting

Predicted vs True Values for GradientBoostingRegressor [Sales]

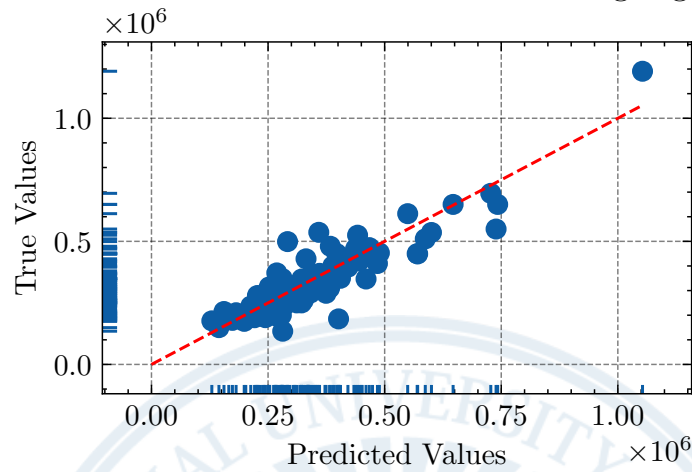


Figure 26: Predicted versus true apartment sales prices using Gradient Boosting

Predicted vs True Values for GradientBoostingRegressor [Rents]

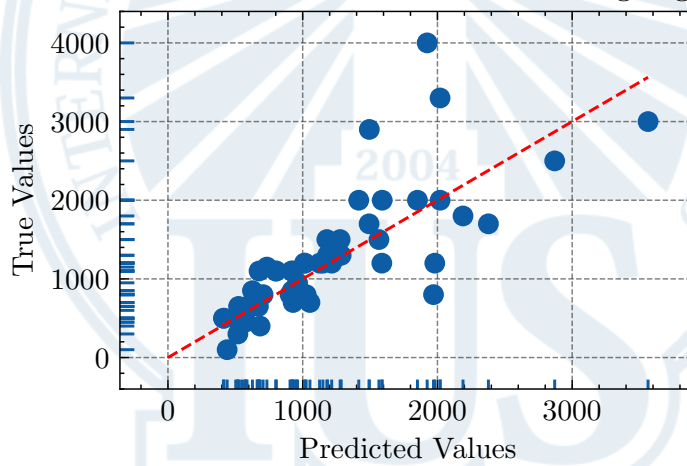


Figure 27: Predicted versus true apartment rental prices using Gradient Boosting

3.2.4 Random Forest

Predicted vs True Values for RandomForest [Sales]

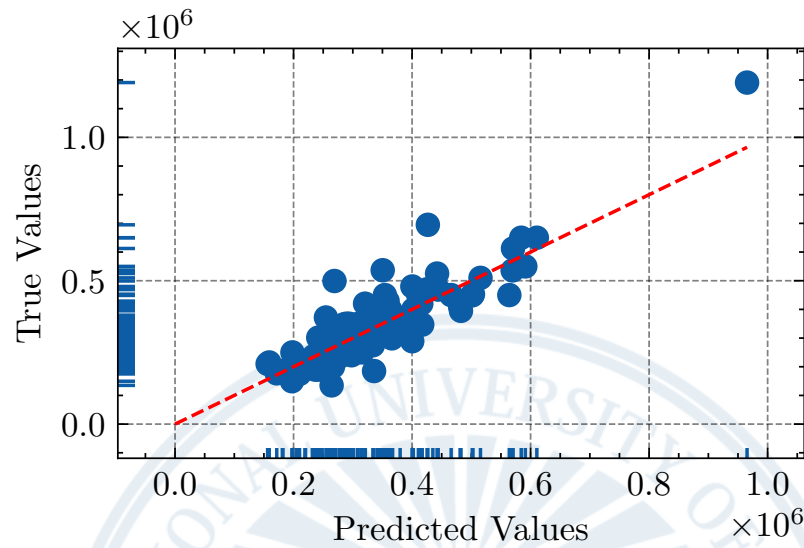


Figure 28: Predicted versus true apartment sales prices using Random Forest

Predicted vs True Values for RandomForest [Rents]

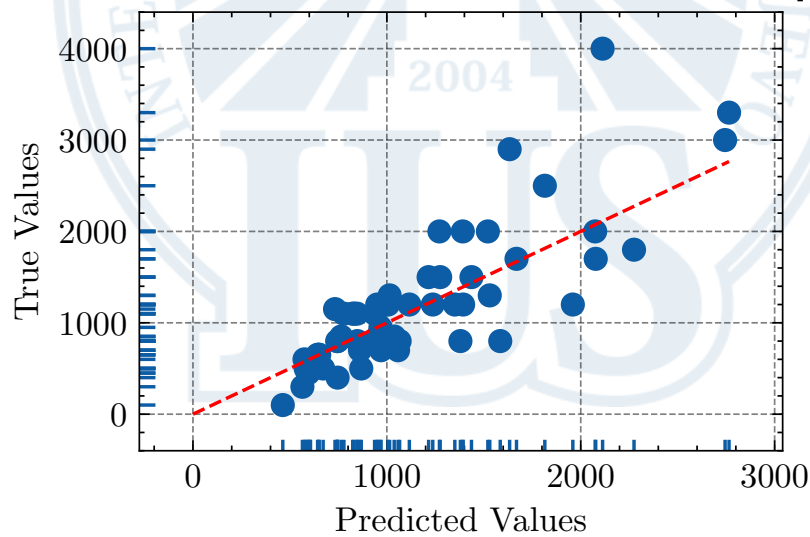


Figure 29: Predicted versus true apartment rental prices using Random Forest

3.2.5 K-Nearest Neighbors

Predicted vs True Values for KNeighborsRegressor [Sales]

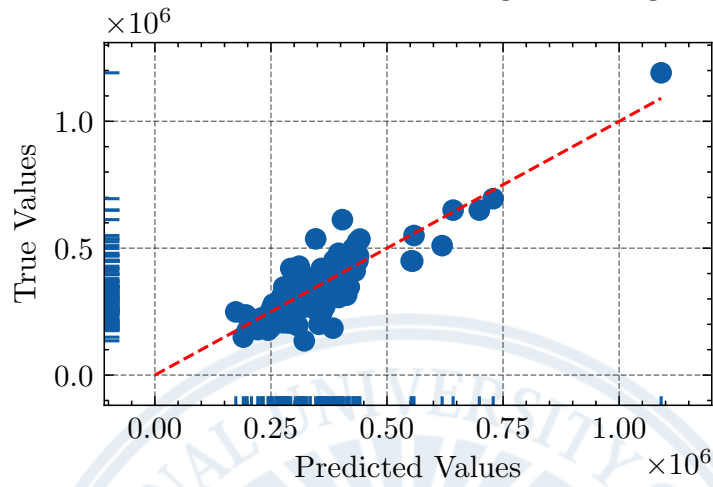


Figure 30: Predicted versus true apartment sales prices using K-Nearest Neighbors

Predicted vs True Values for KNeighborsRegressor [Rents]

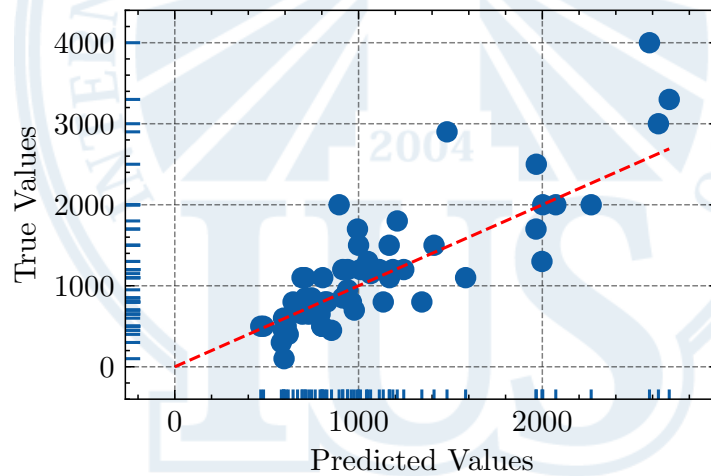


Figure 31: Predicted versus true apartment rental prices using K-Nearest Neighbors

3.3 Model Selection

When it came to model selection we usually had two big criteria:

- 1. The model should generalize well, hence it should perform good on testing data*
- 2. The model should have a smaller error on the test data relative to the baseline error (since the baseline error was taken from the test data)*

This means that we should first look at tables 21 and 24 and pick the ones that had the highest R^2 score as this meant that the model was able to explain most of the variance that appeared in the data that it has never seen before.

If we have had a hard time picking between the models based just on R^2 score, we would then go to tables 22 and 25 and pick out models that had the lowest Mean Absolute Error, because these models made the smallest mistakes and if this wasn't enough we also had tables 23 and 26 to help us make decisions based on the Root Mean Square Error.

And on top of that we had plots of comparing each models Predicted Value to the True Value (Ground Truth).

*For **sales** we decided to go with the Support Vector Machine, and for **rents** we decided to go with K-Nearest Neighbors*

4 Conclusion

This project really opened up our eyes to what people in the profession of Machine Learning actually do, we ended up learning how to use webscraping to gather data, to explore the data to get a better insight, we learned a lot of new libraries such as geo-pandas and OSMNX. This project was also very handy and practical as it also made us think how could we gather information good enough for our model to learn crucial things.

The first time we tried to train machine learning models without spacial data and saw how badly the models generalized we knew we had to somehow find a way to give it more valuable data as we would end up cutting a lot of apartments. Here we really learned that quality is as important if not more than quantity of data. Another example of this was actually even after getting our spacial data in the model did perform much better but for some reason it still was not perform as good as we had hoped for it to do. When we started to look deeper into why this could have happened, we got an idea to simply plot out the apartment to see a general image of the apartments, and this is when we saw apartments that were being sold in Sarajevo but their location marked them to be way out of Sarajevo and sometimes even outside of Bosnia and Herzegovina! So we knew we had to start cleaning up the dataset even more, which did scare us at first because we were already at a small number of apartments and this would end up cutting it even more, but after doing everything we talked about in sections 2.3.1 and 2.3.3 we got an even better result, even though we had a much lower number of apartments then at the start!

There were other things that we also wanted to try out but we just did not have the time. We also wanted to try to possibly use models such as the KNN Imputer to try to impute the missing "room" values.

Another thing that we wanted to do was change the Web Scraping Code to also take out the given address of a place and use OSMNX to try to partially guess the coordinates of the apartments, but sadly we did not have time to change and rerun web scraping as it did take hours to finish.

Overall considering the challenges of working with Bosnia and Herzegovina, mainly the lack of data publicly available, we are quite happy to have done this work especially since there were no other papers covering this exact topic of predicting the housing prices in Bosnia and Herzegovina.

5 Distribution of work

Team member name	Specific tasks completed (should not be ambiguous)
Emre Arapčić-Uevak	Exploratory Data Analysis (statistical summaries, visualizations, missing value analysis) Feature Engineering (feature selection, feature creation, preprocessing pipelines) Machine Learning (model selection, hyperparameter tuning, cross-validation, test evaluation) Performance analysis and result interpretation Preparation of figures, tables, and final report.
Mustafa Sinanović	Web scraping and data collection Assistance with exploratory data analysis Development of a mobile application Integration of the trained machine learning model into the mobile application backend

Table 27: Work Distribution Table

6 Resources

All of the code, documentation, and even the mobile app can be found on our GitHub Repository:

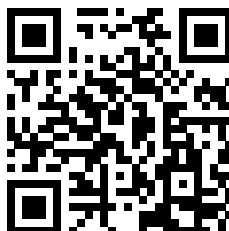


Figure 32: QR Code to our repository